

FIȘĂ DE LUCRU NR. 1

Structuri Liniare de Date: Stiva și Coda

Nume și prenume: _____

Clasa: _____

Data: _____



Obiective de învățare:

- ☐ Să identifice caracteristicile stivei și cozii ca modele conceptuale liniare
- ☐ Să explice principiul LIFO (Last In, First Out) și FIFO (First In, First Out)
- ☐ Să implementeze operațiile de bază cu stive și cozi folosind clasa list din Python
- ☐ Să aplice structurile potrivite în contexte reale

PARTE TEORETICĂ

1. Modelul conceptual liniar — Lista

O listă este o colecție ordonată de elemente în care fiecare element (cu excepția primului și ultimului) are exact un predecesor și un succesor. Elementele pot fi accesate secvențial sau direct, în funcție de tipul listei.

2. Stiva (Stack) — principiul LIFO

Definiție: O stivă este o structură liniară în care inserarea și extragerea elementelor se fac doar la un singur capăt, numit vârful stivei. Ultimul element adăugat este primul care se extrage (LIFO — Last In, First Out).

Analog din viața reală: O stivă de farfurii — adaugi farfurii deasupra și le iei tot de deasupra.

Operații principale:

- ☐ push(x) — adaugă elementul x în vârful stivei
- ☐ pop() — extrage și returnează elementul din vârful
- ☐ peek() / top() — consultă elementul din vârful fără a-l extrage
- ☐ is_empty() — verifică dacă stiva este goală

Implementare în Python folosind list:

```
# Stiva folosind list Python
stiva = [] # Stiva vida

# Operatia push
stiva.append(10) # stiva = [10]
stiva.append(20) # stiva = [10, 20]
stiva.append(30) # stiva = [10, 20, 30]

# Operatia pop
element = stiva.pop() # element = 30, stiva = [10, 20]
```

```
# Consultare varf (peek)
varf = stiva[-1]          # varf = 20

# Verificare daca stiva e goala
if len(stiva) == 0:
    print("Stiva este goala!")
else:
    print("Varful stivei:", stiva[-1])
```

3. Coadă (Queue) — principiul FIFO

Definiție: O coadă este o structură liniară în care elementele se adaugă la un capăt (coada) și se extrag de la celălalt capăt (capul). Primul element adăugat este primul extras (FIFO — First In, First Out).

Analog din viața reală: O coadă la casă — primul venit, primul servit.

Operații principale:

- ❑ enqueue(x) — adaugă elementul x la coada listei
- ❑ dequeue() — extrage și returnează elementul din capul cozii
- ❑ front() — consultă primul element fără a-l extrage
- ❑ is_empty() — verifică dacă coada este goală

Implementare în Python folosind list:

```
# Coadă folosind list Python
coada = []          # Coadă vida

# Operatia enqueue (adaugare la coada)
coada.append("Ana")  # coada = ["Ana"]
coada.append("Bogdan") # coada = ["Ana", "Bogdan"]
coada.append("Carla") # coada = ["Ana", "Bogdan", "Carla"]

# Operatia dequeue (extragere din fata)
primul = coada.pop(0) # primul = "Ana", coada = ["Bogdan", "Carla"]

# Consultare primul element
cap = coada[0]        # cap = "Bogdan"

print("Coadă:", coada)
```

4. Tabel comparativ: Stivă vs. Coadă

Criteriu	Stivă (Stack)	Coadă (Queue)
Principiu	LIFO (Last In, First Out)	FIFO (First In, First Out)
Inserare	La vârful — append(x)	La coadă — append(x)
Extragere	Din vârful — pop()	Din cap — pop(0)
Exemplu real	Stivă de farfurii, funcția Undo	Coadă la ghișeu, tipărire documente

EXERCIȚII

Exercițiul 1 — Recunoaștere (10 puncte)

Identifică dacă fiecare situație descrisă mai jos corespunde unei stive, cozi sau liste simple. Motivează răspunsul.

- ☐ a) Browserul web pastreaza paginile vizitate, iar butonul Inapoi deschide ultima pagina accesata.

Răspuns: _____ Motivație: _____

- ☐ b) Pachetele ajunse la un depozit sunt procesate în ordinea sosirii.

Răspuns: _____ Motivație: _____

- ☐ c) Profesorul anunță notele elevilor în ordinea catalogului (accesare directă).

Răspuns: _____ Motivație: _____

Exercițiul 2 — Trasarea execuției (20 puncte)

Urmărește secvența de instrucțiuni de mai jos și completează starea stivei după fiecare operație:

```
stiva = []
stiva.append(5)
stiva.append(3)
stiva.append(8)
x = stiva.pop()
stiva.append(1)
y = stiva.pop()
```

Instrucțiune executată	Starea stivei	Variabile
stiva = []	[]	—
stiva.append(5)		
stiva.append(3)		
stiva.append(8)		
x = stiva.pop()		x = ____
stiva.append(1)		
y = stiva.pop()		y = ____

Exercițiul 3 — Programare (30 puncte)

Cerință: Scrie un program Python care simulează o coadă de așteptare la o clinică medicală. Programul va:

- ☐ Adăuga 5 pacienți (cu nume introduse de la tastatură) în coadă
- ☐ Afișa pacienții în ordinea în care vor fi consultați
- ☐ Extrage și afișa câte un pacient la fiecare apăsare a tastei Enter

Spațiu pentru rezolvare:

Exercițiul 4 — Analiză și Evaluare (20 puncte)

Un coleg a scris codul de mai jos pentru a gestiona apeluri telefonice la un call center. Identifică eroarea de logică și propune corectura.

```
apeluri = []
apeluri.append("Client A - 10:05")
apeluri.append("Client B - 10:07")
apeluri.append("Client C - 10:12")

# Operatorul dorește să prelucreze apelurile în ordinea sosirii
while len(apeluri) > 0:
    urmator = apeluri.pop()    # <-- posibila eroare
    print("Se prelucrează:", urmator)
```

a) Care este eroarea?

b) Scrie codul corect:

Notare orientativă:

- ☐ Exercițiul 1: 10 puncte
- ☐ Exercițiul 2: 20 puncte
- ☐ Exercițiul 3: 30 puncte
- ☐ Exercițiul 4: 20 puncte
- ☐ 10 puncte din oficiu

Total: 90 + 10 puncte din oficiu = 100 puncte